COLLEGE LABORATORY REPORT

Lab 5: Static Code Analysis

Project: School Bus Transport Register

Programming language	Python 3
Analysis tools	Pylint, Bandit and Flake8
GitHub repository	https://github.com/NiteshMogaveer777/L5_24_school_transport

Submitted by (click a field to edit)

Student name	[Student Name]
Register number	[Register Number]
Department	[Department]
College	[College Name]
Faculty name	[Faculty Name]
Submission date	[DD/MM/YYYY]

Personal and college details were not supplied, so they are left as editable placeholders.

Table of Contents

1. Cover Page . . . 3
2. Abstract . . . 3
3. Objectives . . . 3
4. Introduction . . . 3
5. Project Description . . . 4
6. Tools and Technologies . . . 4
7. Methodology and Implementation . . . 4
Step 1: Open the repository and project in VS Code . . . 4
Step 2: Create and activate a virtual environment . . . 5
Step 3: Install the tools . . . 5
Step 4: Run the original program . . . 5
Steps 5 and 6: Initial static analysis and saving the before-reports . . . 5
Step 7: Identify and document issues . . . 5
Step 8: Correct the source code . . . 5
Steps 9 and 10: Re-run all three tools and save the after-reports . . . 5
Step 11: Test the corrected program . . . 5
Step 12: Commit and push to GitHub . . . 5
8. Static Analysis Results . . . 6
8.1 Pylint . . . 6
8.2 Bandit . . . 6
8.3 Flake8 . . . 7
9. Issues Identified and Fixes . . . 7
10. Before-and-After Comparison . . . 8
11. Testing and Output . . . 8
12. Git and GitHub . . . 9
13. Learning Outcomes . . . 9
14. Conclusion . . . 10
15. References . . . 10
Appendix A: Original Source Code (commit f689f32) . . . 11
Appendix B: Corrected Source Code (current main) . . . 12

Scope and Evidence Basis

This report is built only from evidence that could actually be inspected. The table below states what was and was not available, so every later claim can be traced to a source.

Item	Status
GitHub repository	**Accessed.** Cloned in full; source code, `issues.md`, `reflection.md`, `README.md`, six report files and the complete commit history were read.
Lab 5 handout PDF	**Not supplied.** No file was attached to this request. The repository `README.md` summarises the handout (steps, deliverables, 4 reflection questions, Pylint target) and was used as the substitute authority. Objectives in Section 3 are derived from it. Please compare against the original handout before submission.
Analysis reports	**Accessed.** `reports/` holds before and after output for all three tools (Pylint, Bandit, Flake8).
Console output / screenshots	**Not supplied.** No student screenshots exist in the repository. Console output in Section 11 was reproduced by re-running the committed code and tools during preparation of this report, and is labelled as such.
Pylint 9.68/10	**Not found.** No report in the repository shows 9.68/10. The committed `pylint_after.txt` shows **10.00/10**, which an independent re-run also confirmed. This report therefore uses 10.00/10. See Section 10.

1. Cover Page

The cover page (page preceding this table of contents) gives the lab title, project, language, tools, the clickable repository link, and editable fields for student, register number, department, college, faculty and date.

2. Abstract

This experiment applies static code analysis to a small Python program, the *School Bus Transport Register*, using three tools: Pylint (code quality), Bandit (security) and Flake8 (style). The original program worked but contained problems in all three areas. Before correction, Pylint reported 18 messages and a score of 4.00/10, Bandit reported 4 findings (1 High, 1 Medium, 2 Low), and Flake8 reported 9 style violations.

The issues included an SQL injection risk from string-built queries, a command-execution risk from `subprocess` with `shell=True`, a bare exception handler, unsafe file handling, camel-case function names, an unused argument, missing docstrings and line-length and spacing errors. After the code was corrected, Pylint reported 10.00/10, Flake8 reported no findings, and Bandit reported a single Low-severity finding (use of `assert`), which remains. The program still produces the same demonstration output. The work shows why analysing code before it is deployed improves security, readability and maintainability.

3. Objectives

Based on the repository README, which summarises the Lab 5 handout, the objectives were to:

- run the supplied program first and read its output carefully;
- install and run Pylint, Bandit and Flake8, and save their reports;
- document issues in a table (issue, type, line, description, fix approach), listing high and medium severity first;
- fix at least four documented issues and re-run all three tools to confirm they are resolved, while keeping the program working;
- answer the four reflection questions on static analysis, tool purposes, issues fixed and software quality;
- aim for a Pylint score above 9.5 on the final file; and
- be able to explain the bugs found and the fixes made.

4. Introduction

Static code analysis examines source code *without running it*. Automated tools read the code, compare it with rules and known bad patterns, and report possible defects, security weaknesses and style violations. Because nothing is executed, problems can be found early, when they are cheapest to fix.

Four qualities matter in any program. **Code quality** means the code is correct and free from avoidable defects. **Security** means that untrusted input cannot be used to attack the program or its data. **Readability** means other people can understand the code quickly, which depends on consistent naming, formatting and documentation. **Maintainability** means the code can be changed safely later. Static analysis supports all four by giving objective, repeatable feedback instead of relying on personal opinion.

5. Project Description

The School Bus Transport Register keeps a register of students travelling on school bus routes. It uses an in-memory SQLite database with one table, `riders`, with columns `id`, `student`, `route`, `stop` and `fee`. The repository contains a single source file, `school_transport.py`. Its functions were checked against the corrected source code:

Function	What it does (verified in source)	Feature
create_connection()	Creates the in-memory SQLite database and the `riders` table.	Setup
enroll_rider(student, route, stop, fee)	Inserts a student on a route using a parameterised INSERT and commits.	Enrolling students on bus routes
route_roster(route)	Returns a list of student names on the given route.	Displaying route rosters
export_roster(path, route)	Writes each name on the route to a UTF-8 text file, one per line.	Exporting a route roster to a file
dispatch_alert(route)	Prints `dispatching bus <route>`.	Displaying dispatch alerts
fee_collected(route)	Returns `SUM(fee)` for the route, or 0 if there are no riders.	Calculating route fee totals
main()	Demonstration: enrolls three students, checks R1 total is 3600, exports R1 roster, prints alert, prints R2 roster.	Entry point

The demonstration enrols Tara and Kabir on route R1 (fee 1800 each) and Zoya on route R2 (fee 2100). All five features listed in the assignment brief are implemented in the source.

6. Tools and Technologies

Tool	Purpose
Python 3	The programming language of the project. Provides the `sqlite3` module used for the database and `venv` for isolated environments.
Pylint	Checks code quality: naming conventions, missing docstrings, unused arguments and variables, bad exception handling, resource handling and a 0 to 10 quality score. Messages are grouped as Convention (C), Refactor (R), Warning (W) and Error (E).
Bandit	A security linter for Python. Looks for common weaknesses such as SQL injection, shell injection, hard-coded secrets and unsafe functions. Each finding has a severity and a confidence level, and maps to a CWE identifier.
Flake8	A style-guide checker that combines pycodestyle (PEP 8 layout), pyflakes (logical errors such as unused imports) and a complexity checker. Reports line length, whitespace and blank-line errors.
Git and GitHub	Git records versions of the project as commits; GitHub hosts the repository online so work can be shared, reviewed and submitted.

7. Methodology and Implementation

The procedure below follows the order requested for the lab. Commands are shown for Windows PowerShell, which is what the saved reports indicate was used (the report files are UTF-16 encoded and use `.\school_transport.py` paths, which is typical of PowerShell redirection). Steps 1 to 3 and 12 are standard procedure; no screenshots of them were available to verify the exact commands typed.

Step 1: Open the repository and project in VS Code

```
$ git clone https://github.com/NitheshMogaveer777/L5_24_school_transport
$ cd L5_24_school_transport
$ code .
```

Step 2: Create and activate a virtual environment

The repository's `.gitignore` excludes `.venv/` and `__pycache__/`, so the environment is kept out of version control.

```
$ python -m venv .venv
$ .venv\Scripts\Activate.ps1 # Windows PowerShell
# source .venv/bin/activate # Linux / macOS
```

Step 3: Install the tools

```
$ pip install pylint bandit flake8
```

Step 4: Run the original program

The original file was run first, as the handout requires, to confirm that it works before it is analysed.

```
$ python school_transport.py
```

Steps 5 and 6: Initial static analysis and saving the before-reports

```
$ pylint school_transport.py > pylint_report.txt
$ bandit -r school_transport.py > bandit_report.txt
$ flake8 school_transport.py > flake8_report.txt
```

These are the commands given in the README. In the repository the saved output appears as `reports/pylint_before.txt`, `bandit_before.txt` and `flake8_before.txt` (Bandit timestamp 2026-10-14 16:27 UTC).

Step 7: Identify and document issues

The findings were recorded in `issues.md` as a table of seven issues, ordered with security issues first (SQL injection, unsafe shell execution) as the README requires.

Step 8: Correct the source code

The program was edited to remove the issues (details in Sections 8 and 9). The commit `c68997e` "Fix code quality issues" contains this change to `school_transport.py`.

Steps 9 and 10: Re-run all three tools and save the after-reports

```
$ pylint school_transport.py
$ bandit -r school_transport.py
$ flake8 school_transport.py
```

Output was saved as `reports/pylint_after.txt`, `bandit_after.txt` and `flake8_after.txt` (Bandit timestamp 2026-10-14 16:36 UTC).

Step 11: Test the corrected program

```
$ python school_transport.py
```

Step 12: Commit and push to GitHub

```
$ git add .
$ git commit -m "Fix security issues and document findings"
$ git push origin main
```

The commit message above is the real one recorded in the history. The branch name `main` is assumed; it was not independently confirmed.

8. Static Analysis Results

8.1 Pylint

What it checks. Code quality, naming style, docstrings, unused arguments, exception handling and resource handling, and it gives a score out of 10.

Findings in the original code. Pylint reported 18 messages (14 Convention, 3 Warning, 1 Refactor) and rated the code **4.00/10**.

ID	Message	Original line(s)	Count
C0301	line-too-long (110/100 and 116/100)	5, 9	2
C0114	missing-module-docstring	1	1
C0116	missing-function-docstring	8, 13, 17, 23, 27, 34	6
C0103	invalid-name (enrollRider, routeRoster, exportRoster, dispatchAlert, feeCollected)	8, 13, 17, 23, 27	5
W0613	unused-argument ('driver')	13	1
W1514	unspecified-encoding (open without encoding)	18	1
R1732	consider-using-with	18	1
W0702	bare-except (no exception type specified)	30	1

Pylint messages in the original code (source: reports/pylint_before.txt).

Corrections made. Function names were changed to snake_case; a module docstring and function docstrings were added; the unused `driver` argument was removed; the file is opened with `with open(..., encoding="utf-8")`; the bare `except` was removed; and long lines were wrapped.

Results after corrections. `pylint_after.txt` contains no messages and the line "Your code has been rated at 10.00/10 (previous run: 4.00/10, +6.00)". This was confirmed by an independent re-run on the committed file.

8.2 Bandit

What it checks. Security weaknesses. Each finding carries a test ID, severity, confidence and a CWE reference.

Findings in the original code. Four findings: 1 High, 1 Medium, 2 Low (confidence: 3 High, 1 Medium), scanned over 30 lines of code.

Test ID	Severity / Confidence	CWE	Line	Description
B602	High / High	CWE-78	24	subprocess call with shell=True
B608	Medium / Medium	CWE-89	9	Possible SQL injection through string-built query
B404	Low / High	CWE-78	1	Import of subprocess module
B101	Low / High	CWE-703	38	Use of assert

Bandit findings in the original code (source: reports/bandit_before.txt).

Corrections made. The INSERT statement now uses `?` placeholders with a tuple of values. The `subprocess` import and call were removed and the alert is printed directly. This removed B608, B602 and B404.

Results after corrections. One finding remains: **B101 (assert used), Low severity, High confidence**, at line 73 of the corrected file, in the demonstration check `assert fee_collected("R1") == 3600`. Totals: High 0, Medium 0, Low 1. This finding was not fixed and is reported as it stands. The assert

is only a self-check in the demonstration code, so its security impact is low, but it would be silently skipped under `python -O`.

8.3 Flake8

What it checks. PEP 8 style (pycodestyle), simple logical errors (pyflakes) and code complexity.

Findings in the original code. Nine violations:

Code	Description	Line(s)	Count
E501	Line too long (> 79 characters)	5 (110), 9 (116), 14 (95), 29 (98)	4
E231	Missing whitespace after ','	8 (columns 24, 30, 35)	3
E302	Expected 2 blank lines, found 1	23	1
E722	Do not use bare 'except'	30	1

Flake8 findings in the original code (source: reports/flake8_before.txt).

Corrections made. Long lines were wrapped (the SQL text was moved to multi-line strings), spaces were added after commas in the signature, blank lines were corrected, and the bare `except` was removed.

Results after corrections. `flake8_after.txt` is empty (0 bytes), meaning no violations. A re-run on the committed file also reported none.

9. Issues Identified and Fixes

Each row lists what the tools detected and what the code diff shows was changed. Only issues supported by the code or reports are included. Rows 1 to 7 correspond to the seven rows of the student's `issues.md`; rows 8 and 9 come from the tool reports.

Issue	Type	Original Code Problem	Fix Applied	Verification
1. SQL injection risk	Security (Medium)	`enrollRider` built the INSERT with an f-string, putting student, route and stop text straight into the SQL (line 9).	Query rewritten with ? placeholders and a values tuple.	Bandit B608 present before; absent after.
2. Unsafe shell execution	Security (High)	`subprocess.run("echo dispatching bus " + route, shell=True)` (line 24) lets a crafted route value run shell commands.	`subprocess` removed; alert printed with `print(f"dispatching bus {route}")`.	Bandit B602 and B404 present before; absent after. Program still prints *dispatching bus R1*.
3. Broad (bare) exception	Bug / Quality	`feeCollected` wrapped the query in `try ... except:` (line 30), hiding any error.	Handler removed; `result[0] or 0` handles an empty route.	Pylint W0702 and Flake8 E722 present before; absent after.
4. File handling	Quality	`open(path, "w")` with manual `close()`, no encoding (lines 18-21).	`with open(path, "w", encoding="utf-8")`.	Pylint W1514 and R1732 before; absent after. Output file verified.
5. Function naming	Style	Five camelCase names: enrollRider, routeRoster, exportRoster, dispatchAlert, feeCollected.	Renamed to enroll_rider, route_roster, export_roster, dispatch_alert, fee_collected.	Pylint C0103 x5 before; absent after.
6. Unused argument	Quality	`routeRoster(route, driver)` never used `driver`; callers passed `None` or "Mr. Das".	Parameter removed from definition and all calls.	Pylint W0613 before; absent after.
7. Missing docstrings	Quality	No module docstring and no function docstrings.	Module docstring and a docstring on every function added.	Pylint C0114 and C0116 x6 before; absent after.
8. Formatting issues	Style	Lines up to 116 characters; no space after commas; one blank line before a function.	Lines wrapped; spacing and blank lines corrected.	Flake8 E501 x4, E231 x3, E302 x1 and Pylint C0301 x2 before; absent after.

Issue	Type	Original Code Problem	Fix Applied	Verification
9. Use of assert	Security (Low)	`assert` used for a check in `main()`.	**Not fixed.** Still present at line 73.	Bandit B101 reported both before and after.

Additional refactoring seen in the diff but not flagged by any tool: SQL table creation was moved into a `create_connection()` function, and `main()` is now called only under `if __name__ == "__main__":` (originally it ran on every import). Behaviour change: for a route with no riders, `fee_collected` now returns 0, where the original returned `None` (checked by running both versions).

10. Before-and-After Comparison

Measure	Before	After	Evidence
Pylint score	4.00/10	10.00/10	Verified (reports and re-run)
Pylint messages	18	0	Verified
Bandit findings	4 (High 1, Medium 1, Low 2)	1 (Low 1)	Verified
Flake8 findings	9	0	Verified
Code readability	camelCase names, no docstrings, lines up to 116 characters	snake_case names, docstrings throughout, lines within limits	Supported by 0 Pylint and 0 Flake8 findings; qualitative otherwise
Security improvements	SQL injection risk, shell execution (High), subprocess import	Parameterised query, no shell call, no subprocess import; only Low B101 left	Verified by Bandit
File handling and maintainability	Manual open/close, no encoding; bare except; unused argument; code ran on import	Context manager with UTF-8; no bare except; no unused argument; entry-point guard; connection setup in a function	Verified by code diff and Pylint; maintainability itself is not measured

On the Pylint score. The initial score was 4.00/10, as recorded in `pylint_before.txt`. The brief suggested that a later report showed 9.68/10, but **no 9.68/10 value appears in any report or commit in the repository**, so it is not claimed here. The committed after-report shows **10.00/10** and an independent re-run on the same file confirmed it. The README target of above 9.5 is therefore met. Note that 10/10 from Pylint does not mean a perfect program: Bandit still reports one Low finding and the program has no unit tests.

11. Testing and Output

No student screenshots or terminal captures were supplied, so none are shown. The program was tested as follows:

- **Functional run.** The corrected program was run and the original program run for comparison. Both give the same output.
- **Built-in check.** `main()` asserts that route R1 totals 3600. It passed, because the program exited normally.
- **Exported file.** `route_r1.txt` contained the names Tara and Kabir, one per line.
- **Edge case.** An unknown route returned 0 fees and an empty roster with the corrected code.
- **Tool re-run.** Pylint, Bandit and Flake8 were run again on the committed file.

Console output reproduced during preparation of this report (Python 3.12.3 on Linux; Pylint 4.1.3, Bandit 1.9.4, Flake8 7.4.1). It was not captured from the student's own machine, whose reports used Windows paths:

```
$ python school_transport.py
dispatching bus R1
['Zoya']
$ cat route_r1.txt
Tara
Kabir
$ pylint school_transport.py
Your code has been rated at 10.00/10
$ flake8 school_transport.py
(no output)
$ bandit -r school_transport.py
Issue: [B101:assert_used] ... Location: school_transport.py:73:4
Total issues (by severity): Low: 1 Medium: 0 High: 0
```

12. Git and GitHub

Repository: https://github.com/NiteshMogaveer777/L5_24_school_transport

Version control keeps a history of every change, so the original code can always be compared with the corrected version, and the work is stored safely on GitHub for submission. The history below was read directly from the cloned repository:

Commit	Author	Date	Message (as recorded)	Content
f689f32	Janya Mahesh	2026-10-13	Add Lab 5 static analyis scenario	Starter files: school_transport.py (original), README.md, .gitignore
c68997e	Nithesh	2026-10-14	Fix code quality issues	school_transport.py rewritten (64 insertions, 27 deletions)
c302b77	Nithesh	2026-10-14	Fix security issues and document findings	Added issues.md and six report files in reports/
cea8591	Nithesh	2026-10-14	Add reflection and analysis reports	Added reflection.md

The first commit is the instructor's starter code; the later three are the student's submission work, from the original code to a completed set of deliverables. Commit messages and report timestamps are the real ones. One observation: the commit titled "Fix code quality issues" changes the source file, which also contains the security fixes, so the commit message does not describe the SQL and shell fixes. The spelling "analyis" in the first commit message is as recorded.

13. Learning Outcomes

- **Static code analysis.** Problems can be found without running the program, and different tools find different kinds of problems: Pylint found quality issues, Bandit security issues and Flake8 layout issues. Some issues, such as the bare except, were reported by two tools.
- **Secure SQL queries.** Building SQL with an f-string lets input change the meaning of the query. Parameterised queries with `?` placeholders keep data separate from SQL. Interestingly, the original `routeRoster` and `feeCollected` already used placeholders; only the INSERT was unsafe.
- **Python naming and documentation conventions.** Functions use snake_case; modules and functions should have docstrings; unused parameters should be removed.
- **Safe file handling.** Using `with open(..., encoding="utf-8")` closes the file even if an error occurs and avoids platform-dependent encoding.
- **Code quality tools.** How to install, run, read and compare tool reports, and that a perfect Pylint score is not proof of complete safety (Bandit still reports a Low item).

- **Version control.** Committing in logical steps, keeping reports as evidence, and using `.gitignore` to keep the virtual environment out of the repository.

14. Conclusion

Static analysis of the School Bus Transport Register with Pylint, Bandit and Flake8 exposed 18 quality messages, 4 security findings and 9 style violations. By fixing the SQL injection risk, removing the shell call, replacing the bare exception handler, using safe file handling, renaming functions, removing an unused argument, adding docstrings and fixing formatting, the Pylint score rose from 4.00/10 to 10.00/10, Flake8 findings fell from 9 to 0, and Bandit findings fell from 4 to 1 (Low severity, `assert` in demonstration code). The program's visible behaviour was unchanged. The lab shows that static analysis is a fast, low-cost way to improve security, readability and maintainability, and that it works best alongside testing and code review.

15. References

[1] Lab 5: Static Code Analysis, laboratory handout (PDF). Not supplied for this report; cite the original file as provided by the faculty. Summarised in the repository README.

[2] Pylint documentation. https://pylint.readthedocs.io/

[3] Bandit documentation. https://bandit.readthedocs.io/

[4] Flake8 documentation. https://flake8.pycqa.org/

[5] Python Software Foundation. Python documentation, *sqlite3* module. https://docs.python.org/3/library/sqlite3.html

[6] School Bus Transport Register (L5_24_school_transport), GitHub repository. https://github.com/NiteshMogaveer777/L5_24_school_transport

Appendix A: Original Source Code (commit f689f32)

```
  1  import subprocess
  2  import sqlite3
  3
  4  conn = sqlite3.connect(":memory:")
  5  conn.execute("CREATE TABLE riders (id INTEGER PRIMARY KEY, student TEXT, route TEXT, stop TEX
T, fee INTEGER)")
  6
  7
  8  def enrollRider(student,route,stop,fee):
  9      conn.execute(f"INSERT INTO riders (student, route, stop, fee) VALUES ('{student}', '{rout
e}', '{stop}', {fee})")
 10      conn.commit()
 11
 12
 13  def routeRoster(route, driver):
 14      return [r[0] for r in conn.execute("SELECT student FROM riders WHERE route = ?", (route,)
)]
 15
 16
 17  def exportRoster(path, route):
 18      out = open(path, "w")
 19      for name in routeRoster(route, None):
 20          out.write(name + "\n")
 21      out.close()
 22
 23  def dispatchAlert(route):
 24      subprocess.run("echo dispatching bus " + route, shell=True, check=False)
 25
 26
 27  def feeCollected(route):
 28      try:
 29          return conn.execute("SELECT SUM(fee) FROM riders WHERE route = ?", (route,)).fetchone
()[0]
 30      except:
 31          return 0
 32
 33
 34  def main():
 35      enrollRider("Tara", "R1", "Lake Road", 1800)
 36      enrollRider("Kabir", "R1", "Mill Street", 1800)
 37      enrollRider("Zoya", "R2", "Fort Gate", 2100)
 38      assert feeCollected("R1") == 3600
 39      exportRoster("route_r1.txt", "R1")
 40      dispatchAlert("R1")
 41      print(routeRoster("R2", "Mr. Das"))
 42
 43
 44  main()
```

Appendix B: Corrected Source Code (current main)

```python
"""School bus transport register."""
import sqlite3


def create_connection():
    """Create the riders database."""
    connection = sqlite3.connect(":memory:")
    connection.execute(
        """
        CREATE TABLE riders (
            id INTEGER PRIMARY KEY,
            student TEXT,
            route TEXT,
            stop TEXT,
            fee INTEGER
        )
        """
    )
    return connection


conn = create_connection()


def enroll_rider(student, route, stop, fee):
    """Enroll a student on a bus route."""
    conn.execute(
        """
        INSERT INTO riders (student, route, stop, fee)
        VALUES (?, ?, ?, ?)
        """,
        (student, route, stop, fee),
    )
    conn.commit()


def route_roster(route):
    """Return the names of students on a route."""
    result = conn.execute(
        "SELECT student FROM riders WHERE route = ?",
        (route,),
    )
    return [row[0] for row in result]


def export_roster(path, route):
    """Export a route roster to a text file."""
    with open(path, "w", encoding="utf-8") as output_file:
        for name in route_roster(route):
            output_file.write(name + "\n")


def dispatch_alert(route):
    """Print a bus dispatch alert."""
    print(f"dispatching bus {route}")


def fee_collected(route):
    """Calculate the total fees for a route."""
    result = conn.execute(
        "SELECT SUM(fee) FROM riders WHERE route = ?",
        (route,),
    ).fetchone()
    return result[0] or 0


def main():
    """Run the transport register demonstration."""
    enroll_rider("Tara", "R1", "Lake Road", 1800)
```

```
70      enroll_rider("Kabir", "R1", "Mill Street", 1800)
71      enroll_rider("Zoya", "R2", "Fort Gate", 2100)
72
73      assert fee_collected("R1") == 3600
74
75      export_roster("route_r1.txt", "R1")
76      dispatch_alert("R1")
77      print(route_roster("R2"))
78
79
80  if __name__ == "__main__":
81      main()
```

Line 73 contains the remaining `assert` reported by Bandit (B101).